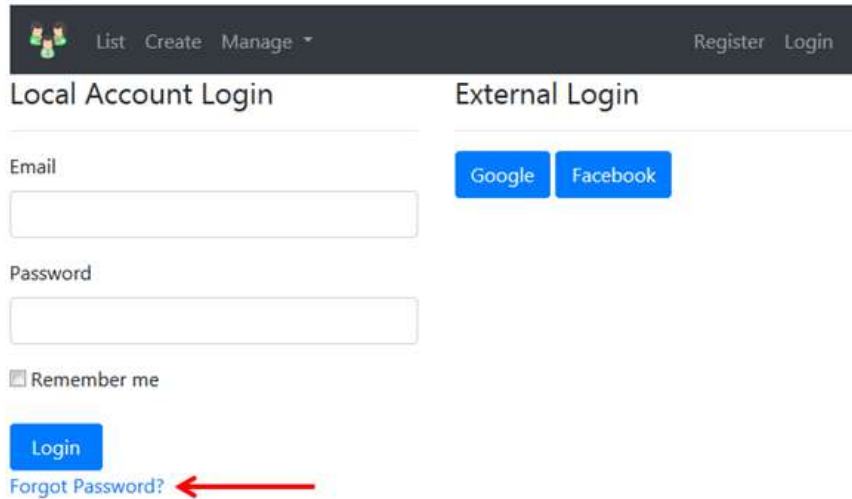


115 - Forgot password

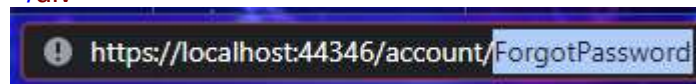
The first step is to include the **Forgot Password** link on the Login view.



The screenshot shows a web application interface with a dark header bar containing 'List', 'Create', 'Manage', 'Register', and 'Login' links. Below the header, there are two main sections: 'Local Account Login' and 'External Login'. The 'Local Account Login' section has input fields for 'Email' and 'Password', a 'Remember me' checkbox, and a 'Login' button. Below the 'Login' button is a link labeled 'Forgot Password?' with a red arrow pointing to it. The 'External Login' section has buttons for 'Google' and 'Facebook'.

```
Login.cshtml  [X]
<button type="submit" class="btn btn-primary">Login</button>
<div>
  <a asp-action="ForgotPassword">Forgot Password?</a>
</div>
```

```
<div>
  <a asp-action="ForgotPassword">Forgot Password?</a>
</div>
```



The screenshot shows a browser address bar with the URL `https://localhost:44346/account/ForgotPassword`.

Forgot Password in ASP.NET Core



```
public class ForgotPasswordViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }
}
```

```
@model ForgotPasswordViewModel

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <button type="submit" class="btn btn-primary">Submit</button>
        </form>
    </div>
</div>
```

We just need the user email address to send the password reset link. So the `ForgotPasswordViewModel` class contains just one property - `Email`

```
[Required]
[EmailAddress]
public string Email { get; set; }
```

Forgot Password View

Views->Account -> Razor View

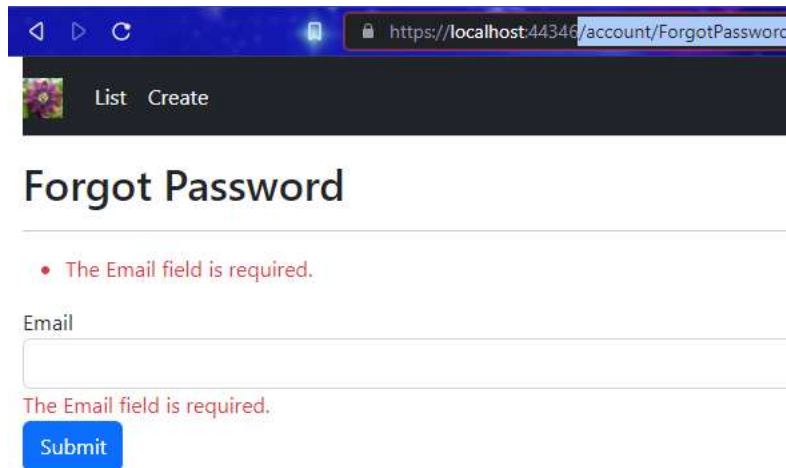
```
@model ForgotPasswordViewModel
```

```
<h2>Forgot Password</h2>
<hr />
<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <button type="submit" class="btn btn-primary">Submit</button>
        </form>
    </div>
</div>
```

Forgot Password Action Methods

Include the following `HTTP GET` and `POST ForgotPassword()` actions in the `Account` controller. The code in the actions is commented and self-explanatory.

```
[HttpGet]
[AllowAnonymous]
public IActionResult ForgotPassword()
{
    return View();
}
```



Forgot Password in ASP.NET Core

Generate Password Reset Token

```
var token = await userManager.GeneratePasswordResetTokenAsync(user);
```

service method (passing object)

1

Build Password Reset Link

```
var passwordResetLink = Url.Action("ResetPassword", "Account",
    new { email = model.Email, token = token }, Request.Scheme);
```

action controller URL http, https+

2

Log Password Reset Link

```
logger.Log(LogLevel.Warning, passwordResetLink);
```

record C:\DemoLogs

3

Send User to Forgot Password Confirmation View

```
return View("ForgotPasswordConfirmation");
```

4

Forgot Password Action

```
[HttpPost][AllowAnonymous]
public async Task<IActionResult> ForgotPassword(ForgotPasswordViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = await userManager.FindByEmailAsync(model.Email);
        if (user != null && await userManager.IsEmailConfirmedAsync(user))
        {
            var token = await userManager.GeneratePasswordResetTokenAsync(user);

            var passwordResetLink = Url.Action("ResetPassword", "Account",
                new { email = model.Email, token = token }, Request.Scheme);

            logger.Log(LogLevel.Warning, passwordResetLink);

            return View("ForgotPasswordConfirmation");
        }
        return View("ForgotPasswordConfirmation");
    }
    return View(model);
}
```

```
[HttpPost]
[AllowAnonymous]
public async Task<IActionResult> ForgotPassword(ForgotPasswordViewModel model)
{
    if (ModelState.IsValid)
    {
        // Find the user by email
        var user = await userManager.FindByEmailAsync(model.Email);
        // If the user is found AND Email is confirmed
        if (user != null && await userManager.IsEmailConfirmedAsync(user))
        {
            // Generate the reset password token
            var token = await userManager.GeneratePasswordResetTokenAsync(user);

            // Build the password reset link
            var passwordResetLink = Url.Action("ResetPassword", "Account",
                new { email = model.Email, token = token }, Request.Scheme);

            // Log the password reset link
            logger.Log(LogLevel.Warning, passwordResetLink);

            // Send the user to Forgot Password Confirmation view
            return View("ForgotPasswordConfirmation");
        }

        // To avoid account enumeration and brute force attacks, don't
        // reveal that the user does not exist or is not confirmed
        return View("ForgotPasswordConfirmation");
    }

    return View(model);
}
```




<https://localhost:44346/Account/ResetPassword?email=Anthony@abv.bg&token=CfDJ8NXI8GzyYcNDu0rWycevLizOli%2FTrx4Zy%2BjSE6qOfgkOihqquoM61F8V1VdL6mE89mof8Gx4xUfWjwAYi9Pbiq3QIM0DFsC2Wo3sfxXOHMnZlItkaHM3lfTXF9ocdKxQaMM7OlzaleRWYj6BP2D9kNf8NCUcUD2c%2BFQ%2B29N1cEEuIsIvuLMo6ZRVE1TIpeq87gR9nWljR%2F4UPKTIAe4blJyAZ4XMCKbRb5n49jdYPqwRx>

At the moment, if we try to use the password reset link, we get a **404 error**. This is because we do not have `ResetPassword()` action in the `AccountController`. We will discuss how to implement this in our next video.